

```

from PySide6.QtWidgets import QStyledItemDelegate, QStyle, QApplication, QStyleOptionViewItem
from PySide6.QtGui import QPainter, QFont, QFontMetrics
from PySide6.QtCore import Qt, QSize

class IndexTextFormatterDelegate(QStyledItemDelegate):
    """Layers on top of Column 0 to render LaTeX formatting while preserving tree hierarchy positions."""

    def paint(self, painter, option, index):
        if index.column() == 0:
            raw_text = index.data(Qt.ItemDataRole.DisplayRole)
            if not raw_text:
                super().paint(painter, option, index)
                return

            painter.save()

            # Create a copy of view item state parameters to extract geometry boundaries
            custom_option = QStyleOptionViewItem(option)
            self.initStyleOption(custom_option, index)

            # Render native highlight bounding selection box behind our items
            if custom_option.state & QStyle.StateFlag.State_Selected:
                painter.fillRect(custom_option.rect, custom_option.palette.highlight())
                painter.setPen(custom_option.palette.highlightedText().color())
            else:
                painter.setPen(custom_option.palette.text().color())

            # -----
            # FIX 1: CALCULATE THE ACCURATE TREE TEXT GEOMETRY FIELD BOUNDARY
            # -----
            # Bypasses option.rect.x() which overlaps arrow layers.
            # Queries the Qt style engine to find the exact sub-rect where the text belongs.
            text_rect = QApplication.style().subElementRect(
                QStyle.SubElement.SE_ItemViewItemText,
                custom_option,
                custom_option.widget
            )

            text_segments = self._parse_latex_formatting_segments(str(raw_text))

            # Anchor our horizontal starting position securely to the resolved text boundary box
            current_x = text_rect.x()

            # Compute geometric metrics to center text elements vertically within the row cell bounds
            font_metrics = custom_option.fontMetrics
            text_height = font_metrics.height()
            vertical_padding = (custom_option.rect.height() - text_height) // 2
            y_baseline = custom_option.rect.y() + vertical_padding + font_metrics.ascent()

            base_font = QFont(painter.font())

            for text_chunk, is_italic, is_bold in text_segments:
                if not text_chunk:
                    continue
                font = QFont(base_font)
                if is_italic: font.setItalic(True)
                if is_bold: font.setBold(True)

                painter.setFont(font)
                fm = QFontMetrics(font)

                # Render content blocks only if they sit inside the visible right edge column barrier
                if current_x < text_rect.right():
                    # Clip the chunk length gracefully if it threatens to step outside the text viewport bounding rect
                    available_width = text_rect.right() - current_x
                    display_chunk = text_chunk

                    if fm.horizontalAdvance(text_chunk) > available_width:
                        display_chunk = fm.elidedText(text_chunk, Qt.TextElideMode.ElideRight, available_width)

                    painter.drawText(current_x, y_baseline, display_chunk)
                    current_x += fm.horizontalAdvance(display_chunk)

            painter.restore()
        else:
            super().paint(painter, option, index)

    def sizeHint(self, option, index):
        """
        FIX 2: COMPUTATIONAL SIZE HINT OVERRIDE
        Ensures that bold text expansion rules do not result in clipped string layouts.
        """
        if index.column() == 0:
            raw_text = index.data(Qt.ItemDataRole.DisplayRole)
            if not raw_text:
                return super().sizeHint(option, index)

```

```

custom_option = QStyleOptionViewItem(option)
self.initStyleOption(custom_option, index)

text_segments = self._parse_latex_formatting_segments(str(raw_text))
base_font = QFont(custom_option.font)

total_width = 0
max_height = custom_option.fontMetrics.height()

# Aggregate the visual horizontal footprints of all individual token chunks combined
for text_chunk, is_italic, is_bold in text_segments:
    font = QFont(base_font)
    if is_italic: font.setItalic(True)
    if is_bold: font.setBold(True)
    fm = QFontMetrics(font)
    total_width += fm.horizontalAdvance(text_chunk)
    max_height = max(max_height, fm.height())

# Extract parent padding widths to keep icon buffers intact
text_rect = QApplication.style().subElementRect(
    QStyle.SubElement.SE_ItemViewItemText,
    custom_option,
    custom_option.widget
)
indent_padding = text_rect.x() - custom_option.rect.x()

# Return full layout dimensional width constraints plus visual padding buffer margins
return QSize(total_width + indent_padding + 12, max_height + 4)

return super().sizeHint(option, index)

def _parse_latex_formatting_segments(self, text: str) -> list[tuple[str, bool, bool]]:
    """Tokenizes text blocks into styled chunks, handling nested formatting markers."""
    if not text:
        return []

    segments = []
    idx = 0
    text_len = len(text)

    current_italic = False
    current_bold = False
    accumulated_chars = []

    while idx < text_len:
        if text.startswith(r"\textbf{", idx):
            if accumulated_chars:
                segments.append(("".join(accumulated_chars), current_italic, current_bold))
                accumulated_chars = []
            current_bold = True
            idx += 8
        elif text.startswith(r"\textit{", idx):
            if accumulated_chars:
                segments.append(("".join(accumulated_chars), current_italic, current_bold))
                accumulated_chars = []
            current_italic = True
            idx += 8
        elif text[idx] == '}':
            if accumulated_chars:
                segments.append(("".join(accumulated_chars), current_italic, current_bold))
                accumulated_chars = []
            current_italic = False
            current_bold = False
            idx += 1
        elif text.startswith(r"\string", idx):
            idx += 7
        else:
            accumulated_chars.append(text[idx])
            idx += 1

    if accumulated_chars:
        segments.append(("".join(accumulated_chars), current_italic, current_bold))

    if not segments and text:
        segments.append((text, False, False))

    return segments

```